UNIDAD 2 · SESIÓN 7

Laboratorio Guiado

JPQL y Transacciones con Spring Boot

Consultas seguras y ventas transaccionales en TechStore Perú SAC

Dato	Detalle
Curso	Desarrollo Web Integrado (100000ST61)
Carrera	Ingeniería de Sistemas e Informática / Ingeniería de Software
Unidad	Unidad 2 — Back-end con bases de datos
Tema de sesión	JPQL. Transacciones con Spring Boot
Duración estimada	90 minutos (trabajo individual o en pareja)
Requisito previo	Proyecto TechStore Perú SAC de la Sesión 6 (JPA / Hibernate / CRUD) funcionando

1. Contexto del caso

TechStore Perú SAC continúa creciendo. En la Sesión 6 implementaste las entidades Producto y Categoria con JPA/Hibernate y las operaciones CRUD básicas. Ahora el equipo de TechStore necesita tres capacidades adicionales para su back-end:

- Consultar productos con criterios más específicos (stock bajo, filtrado por categoría) usando JPQL.
- Actualizar precios de forma masiva sin traer todos los productos a memoria.
- Registrar ventas de forma segura: si algo falla a mitad de una venta, el sistema no debe quedar con datos inconsistentes (stock descontado sin una venta registrada, por ejemplo).

En este laboratorio construirás estas tres capacidades paso a paso, sobre el mismo proyecto Spring Boot que ya conoces (paquete base pe.utp.techstore).

2. Objetivos de aprendizaje

Al finalizar este laboratorio, serás capaz de:

- Implementar consultas personalizadas con JPQL y parámetros con nombre en un JpaRepository.
- Construir consultas con JOIN entre entidades relacionadas.
- Implementar operaciones de actualización masiva con @Query y @Modifying.
- Aplicar @Transactional en la capa Service para proteger una operación de negocio crítica.
- Verificar, con Postman, que una transacción se revierte (rollback) correctamente ante un error.

3. Prerrequisitos

Antes de comenzar, verifica que cuentas con lo siguiente:

- El proyecto Spring Boot de TechStore Perú SAC de la Sesión 6, con las entidades Producto y Categoria mapeadas y la conexión a la base de datos activa.
- IntelliJ IDEA (o tu IDE preferido) y Postman instalados.
- Conocimientos previos: anotaciones JPA (@Entity, @Id, @ManyToOne), JpaRepository, capas Repository / Service / Controller.

4. Estructura del proyecto

Trabajarás dentro del paquete base pe.utp.techstore. Al finalizar el laboratorio, tu proyecto tendrá esta estructura (los elementos nuevos están marcados):

```
pe.utp.techstore
├── entity
│   ├── Producto.java          (ya existe)
│   ├── Categoria.java         (ya existe)
│   └── Venta.java             ← NUEVO
├── repository
│   ├── ProductoRepository.java   ← se amplía
│   └── VentaRepository.java      ← NUEVO
├── service
│   ├── ProductoService.java      (ya existe)
│   ├── VentaService.java         ← NUEVO
│   └── impl/
│       ├── ProductoServiceImpl.java   ← se amplía
│       └── VentaServiceImpl.java      ← NUEVO
└── controller
    ├── ProductoController.java   ← se amplía
    └── VentaController.java      ← NUEVO
```

5. Parte 1 — Alertas de stock bajo (JPQL con parámetro con nombre)

TechStore quiere un endpoint que le muestre, en cualquier momento, qué productos están a punto de agotarse.

Paso 1.1 — Amplía ProductoRepository

Agrega el siguiente método a la interfaz ProductoRepository, usando un parámetro con nombre:

```
public interface ProductoRepository
        extends JpaRepository<Producto, Integer> {

    @Query("SELECT p FROM Producto p WHERE p.stock < :stockMinimo")
    List<Producto> buscarConStockBajo(
            @Param("stockMinimo") int stockMinimo);
}
```

Paso 1.2 — Expón el endpoint en ProductoController

```
@GetMapping("/stock-bajo")
public List<Producto> obtenerStockBajo(
        @RequestParam(defaultValue = "5") int limite) {
    return productoRepository.buscarConStockBajo(limite);
}
```

Recuerda: como es una consulta de solo lectura, NO necesita @Transactional.

Paso 1.3 — Prueba en Postman

Método	URL	Resultado esperado
GET	localhost:8080/api/productos/stock-bajo?limite=5	Lista de productos con stock menor a 5, código 200 OK
GET	localhost:8080/api/productos/stock-bajo?limite=0	Lista vacía [ ], código 200 OK

6. Parte 2 — Catálogo por categoría (JPQL con JOIN)

La app móvil de TechStore necesita mostrar los productos de una categoría, ordenados del más barato al más caro.

Paso 2.1 — Amplía ProductoRepository

```
@Query("SELECT p FROM Producto p JOIN p.categoria c " +
       "WHERE c.nombre = :nombreCategoria ORDER BY p.precio ASC")
List<Producto> listarPorCategoriaOrdenado(
        @Param("nombreCategoria") String nombreCategoria);
```

Paso 2.2 — Expón el endpoint en ProductoController

```
@GetMapping("/categoria/{nombreCategoria}")
public List<Producto> obtenerPorCategoria(
        @PathVariable String nombreCategoria) {
    return productoRepository.listarPorCategoriaOrdenado(nombreCategoria);
}
```

Paso 2.3 — Prueba en Postman

Método	URL	Resultado esperado
GET	localhost:8080/api/productos/categoria/Electrónica	Productos de "Electrónica" ordenados por precio ascendente
GET	localhost:8080/api/productos/categoria/NoExiste	Lista vacía [ ], código 200 OK

7. Parte 3 — Ajuste masivo de precios (@Query + @Modifying)

El área comercial de TechStore quiere aumentar, de una sola vez, el precio de todos los productos de una categoría en un porcentaje determinado (por ejemplo, por inflación o promoción inversa).

Paso 3.1 — Amplía ProductoRepository

```
@Modifying
@Query("UPDATE Producto p SET p.precio = " +
       "p.precio * (1 + :porcentaje / 100.0) " +
       "WHERE p.categoria.id = :categoriaId")
int aumentarPrecioPorCategoria(
        @Param("categoriaId") Integer categoriaId,
        @Param("porcentaje") double porcentaje);
```

Paso 3.2 — Agrega el método en la capa Service

Recuerda la buena práctica: @Transactional va en el Service, no en el Repository.

```
@Service
public class ProductoServiceImpl implements ProductoService {

    @Override
    @Transactional
    public int aumentarPrecioPorCategoria(Integer categoriaId, double
porcentaje) {
        return productoRepository.aumentarPrecioPorCategoria(categoriaId,
porcentaje);
    }
}
```

Paso 3.3 — Expón el endpoint en ProductoController

```
@PutMapping("/categoria/{categoriaId}/aumentar-precio")
public String aumentarPrecio(
        @PathVariable Integer categoriaId,
        @RequestParam double porcentaje) {
    int filas = productoService.aumentarPrecioPorCategoria(categoriaId,
porcentaje);
    return filas + " producto(s) actualizado(s).";
}
```

Paso 3.4 — Prueba en Postman

Método	URL	Resultado esperado
PUT	localhost:8080/api/productos/categoria/1/aumentar-precio?porcentaje=10	Mensaje "N producto(s) actualizado(s)."; verifica en la BD que los precios subieron 10%

8. Parte 4 — Registrar una venta de forma transaccional

Esta es la parte más importante del laboratorio: vas a construir un flujo de venta que descuenta stock y guarda el registro de venta como una sola unidad. Si el stock no alcanza, NADA debe quedar guardado en la base de datos.

Paso 4.1 — Crea la entidad Venta

```
@Entity
@Table(name = "venta")
public class Venta {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    private Long productoId;
    private int cantidad;
    private LocalDateTime fecha = LocalDateTime.now();

    // constructores, getters y setters

    public Venta() { }

    public Venta(Long productoId, int cantidad) {
        this.productoId = productoId;
        this.cantidad = cantidad;
    }
    // ... getters y setters omitidos por espacio
}
```

Paso 4.2 — Crea VentaRepository

```
public interface VentaRepository extends JpaRepository<Venta, Long> {
}
```

Paso 4.3 — Crea VentaService (interfaz + implementación)

Este es el corazón del laboratorio. Presta atención a cada paso del método:

```
public interface VentaService {
    void registrarVenta(Long productoId, int cantidad);
}
```

```
@Service
public class VentaServiceImpl implements VentaService {

    @Autowired
    private ProductoRepository productoRepository;

    @Autowired
```

```
    private VentaRepository ventaRepository;

    @Override
    @Transactional
    public void registrarVenta(Long productoId, int cantidad) {
        // 1. Buscar el producto
        Producto producto =
productoRepository.findById(productoId.intValue())
                .orElseThrow(() ->
                    new NoSuchElementException("Producto no encontrado"));

        // 2. Validar stock suficiente
        if (producto.getStock() < cantidad) {
            throw new IllegalStateException(
                "Stock insuficiente para el producto: " +
producto.getNombre());
        }

        // 3. Descontar el stock (consulta JPQL de la Sesión 7)
        producto.setStock(producto.getStock() - cantidad);
        productoRepository.save(producto);

        // 4. Registrar la venta
        ventaRepository.save(new Venta(productoId, cantidad));
    }
}
```

Clave del laboratorio:

Si el paso 2 lanza una excepción, Spring Boot revierte automáticamente TODO lo que haya ocurrido antes dentro del método gracias a @Transactional — en este caso, nada porque la excepción ocurre antes de cualquier cambio. Pero si el error ocurriera DESPUÉS de guardar el producto (paso 3) y ANTES de guardar la venta (paso 4), el descuento de stock también se revertiría. Ese es el poder de una transacción: todo o nada.

Paso 4.4 — Crea VentaController

```
@RestController
@RequestMapping("/api/ventas")
public class VentaController {

    @Autowired
    private VentaService ventaService;

    @PostMapping
    public ResponseEntity<String> registrarVenta(
            @RequestParam Long productoId,
            @RequestParam int cantidad) {
        try {
            ventaService.registrarVenta(productoId, cantidad);
            return ResponseEntity.ok("Venta registrada correctamente.");
        } catch (IllegalStateException e) {
            return ResponseEntity.badRequest().body(e.getMessage());
```

```
            }
        }
    }
```

Paso 4.5 — Prueba el caso EXITOSO en Postman

- Elige un producto con stock suficiente (por ejemplo, stock = 10).
- POST localhost:8080/api/ventas?productoId=1&cantidad=2
- Verifica en la base de datos: el stock del producto bajó en 2, y aparece un nuevo registro en la tabla venta.

Paso 4.6 — Prueba el caso de ROLLBACK

Ahora fuerza el escenario de error, para comprobar que la transacción realmente protege los datos:

- Elige un producto y pide una cantidad MAYOR a su stock disponible (por ejemplo, stock = 3, cantidad = 100).
- POST localhost:8080/api/ventas?productoId=1&cantidad=100
- Deberías recibir un 400 Bad Request con el mensaje "Stock insuficiente para el producto: ...".
- Verifica en la base de datos: el stock del producto NO cambió, y NO se creó ningún registro nuevo en la tabla venta.

> **¿Por qué es importante esta prueba?**
> Si tu método NO tuviera @Transactional, y el error ocurriera después de guardar algo, ese cambio parcial quedaría guardado en la base de datos — exactamente el problema del "inventario fantasma" que discutimos al inicio de la sesión.

9. Resumen de endpoints construidos

Método	Endpoint	Propósito
GET	/api/productos/stock-bajo?limite=N	Productos con stock menor al límite (JPQL + parámetro con nombre)
GET	/api/productos/categoria/{nombre}	Catálogo por categoría, ordenado por precio (JPQL + JOIN)
PUT	/api/productos/categoria/{id}/aumentar-precio?porcentaje=N	Ajuste masivo de precios (@Modifying)
POST	/api/ventas?productoId=N&cantidad=N	Venta transaccional con validación de stock y rollback

10. Reto avanzado (opcional)

Si terminaste antes de tiempo, intenta este reto: el gerente de TechStore quiere un reporte con el precio promedio de los productos, agrupado por categoría.

- Implementa en ProductoRepository un método reportePrecioPromedioPorCategoria() usando @Query con GROUP BY (revisa el ejemplo de la sección de Transformación de la clase).
- Expón un endpoint GET /api/reportes/precio-promedio que devuelva el resultado.
- Reto extra: en vez de devolver List<Object[]>, crea una clase DTO (por ejemplo, ReporteCategoriaDTO con los campos categoria y precioPromedio) para que la respuesta JSON sea más clara y legible para el consumidor de la API.

11. Preguntas de reflexión y evaluación

Responde brevemente las siguientes preguntas, con tus propias palabras, al finalizar el laboratorio:

1. ¿Qué diferencia encontraste entre escribir una consulta con EntityManager y con @Query dentro de un JpaRepository? ¿Cuál te resultó más cómoda y por qué?
2. En el método registrarVenta(), ¿qué hubiera pasado si NO se hubiera anotado el método con @Transactional? Describe el escenario problemático que podría ocurrir.
3. ¿Por qué la validación de stock se hace ANTES de modificar cualquier dato, y no después?
4. Explica, con tus propias palabras, qué significa que una transacción sea "atómica", usando el ejemplo de TechStore.
5. ¿En qué otro módulo de tu proyecto integrador (o de un sistema real que conozcas) aplicarías @Transactional? Justifica tu respuesta.

12. Rúbrica de evaluación del laboratorio

Este laboratorio se evalúa sobre 20 puntos, distribuidos de la siguiente manera:

Criterio	Logrado	En proceso	Inicio	Ptos
Consultas JPQL (Partes 1 y 2)	Ambas consultas funcionan correctamente, usan parámetros con nombre y JOIN de forma adecuada.	Al menos una consulta funciona correctamente; la otra presenta errores menores.	Las consultas no compilan o no retornan los resultados esperados.	5
Actualización masiva (Parte 3)	El método @Modifying funciona, está correctamente anotado con @Transactional en el Service.	El método funciona pero @Transactional está mal ubicado (ej. en el Repository).	El endpoint no actualiza los datos o genera error.	4
Transacción de venta (Parte 4)	El flujo completo funciona: venta exitosa se guarda, venta con stock insuficiente revierte todo (verificado en la BD).	El flujo funciona parcialmente: por ejemplo, la venta exitosa funciona pero el rollback no se verificó o no funciona del todo.	El método no implementa validación de stock o no usa @Transactional.	7
Pruebas en Postman	Todos los endpoints fueron probados con evidencia (capturas o colección) de los casos éxito y error.	La mayoría de endpoints fueron probados, falta evidencia de algún caso de error.	No hay evidencia de pruebas o son incompletas.	2
Preguntas de reflexión	Todas las respuestas son claras, usan vocabulario técnico correcto y reflejan comprensión real del tema.	Las respuestas son correctas pero superficiales o incompletas.	Las respuestas no reflejan comprensión del tema o están copiadas de la diapositiva sin adaptación.	2

Puntaje total: 20 puntos.

13. Recursos adicionales

- Documentación oficial de Spring Data JPA — Query Methods y @Query.
- Documentación oficial de Spring Framework — Transaction Management (@Transactional).
- Material de la Sesión 6 (JPA / Hibernate / CRUD) para repasar el mapeo de entidades Producto y Categoria.
- Diapositivas de la Sesión 7: JPQL y Transacciones con Spring Boot.

¡Éxitos con tu laboratorio! Recuerda participar en el foro del curso ante cualquier duda.